Data Preparation

Date	28 October 2026
Team ID	[Insert Team ID]
Project Name	Health360: Comprehensive Healthcare Risk & Disease Analytics
Maximum Marks	5 Marks

Data preparation refers to the process of cleaning, standardizing, transforming, and organizing data so that it is suitable for analysis and reporting. In a Snowflake project, data preparation involves working with the ingested RAW data, identifying data quality issues, creating STAGING tables, applying required transformations, and preparing reliable data for the CURATED and MART layers.

Activity 1: Identifying and Handling Null Values

Identify the NULL or missing values present in the Snowflake dataset, analyze the affected columns, determine the appropriate approach for handling them based on the data and business requirements, apply the required query, and validate the cleaned data to ensure it is ready for further analysis and visualization.

Python code

```
import pandas as pd

df = pd.read_csv("smart_healthcare_dataset.csv")
print("Shape:", df.shape)

# Step 1: NULL audit across all columns
null_counts = df.isnull().sum()
print(null_counts.to_string())
print("Total NULLs:", int(null_counts.sum()))

# Step 2: impute numeric vitals with cohort median (safeguard)
num_cols = df.select_dtypes(include="number").columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Step 3: validate
print("Total NULLs after imputation:", int(df.isnull().sum().sum()))
```

Snowflake SQL equivalent

```
-- Null audit (Snowflake)
SELECT COUNT(*)                          AS total_rows,
       COUNT_IF(blood_pressure IS NULL)  AS null_blood_pressure,
       COUNT_IF(glucose IS NULL)         AS null_glucose,
       COUNT_IF(cholesterol IS NULL)     AS null_cholesterol,
       COUNT_IF(bmi IS NULL)             AS null_bmi
FROM RAW_PATIENT_DATA;

-- Median imputation (safeguard; updates 0 rows when no NULLs exist)
UPDATE RAW_PATIENT_DATA
SET glucose = (SELECT MEDIAN(glucose) FROM RAW_PATIENT_DATA)
WHERE glucose IS NULL;
```

Output

```
Shape: (5000, 16)
age                 0
gender              0
bmi                 0
exercise_level      0
smoking             0
alcohol             0
blood_pressure      0
cholesterol         0
glucose             0
fatigue             0
chest_pain          0
dizziness           0
heart_disease       0
diabetes            0
stroke              0
health_risk_score   0
Total NULLs: 0
Total NULLs after imputation: 0
```

Output Summary: The null audit found no missing values in any of the 16 columns, including blood_pressure, glucose, cholesterol and bmi, so no cells needed imputation. The cohort-median step is kept as a safeguard for future loads, and zero NULLs remain across all 5,000 records.

Activity 2: Duplicate Record Handling

Identify and analyze duplicate records in the Snowflake dataset, determine whether the duplicates are genuine or repeated entries, define the appropriate criteria for identifying duplicates, remove or retain records based on the project requirements, and validate the dataset after treatment to ensure that duplicate records do not affect the accuracy of further analysis and visualizations.

Python code

```
# Step 1: duplicate check on all source columns
# (the dataset has no Patient_ID column, so full-row matching is used)
print("Duplicate rows (all 16 columns):", int(df.duplicated().sum()))

# Step 2: remove duplicates (none expected)
rows_before = len(df)
df = df.drop_duplicates().reset_index(drop=True)
print("Rows before / after:", rows_before, "/", len(df))

# Step 3: add a surrogate patient_id key and confirm uniqueness
df.insert(0, "patient_id", range(1, len(df) + 1))
print("patient_id unique:", df["patient_id"].is_unique)
print("Unique patient rows:", df["patient_id"].nunique())
```

Snowflake SQL equivalent

```
-- Duplicate check on all source columns
SELECT COUNT(*) - (SELECT COUNT(*) FROM (SELECT DISTINCT * FROM RAW_PATIENT_DATA))
       AS duplicate_rows
FROM RAW_PATIENT_DATA;

-- De-duplicated working set
CREATE OR REPLACE TEMPORARY TABLE STG_PATIENT_UNIQUE AS
SELECT DISTINCT * FROM RAW_PATIENT_DATA;
```

Output

```
Duplicate rows (all 16 columns): 0
Rows before / after: 5000 / 5000
patient_id unique: True
Unique patient rows: 5000
```

Output Summary: 0 duplicate patient records were found. The dataset has no Patient_ID column, so duplicates were tested on all 16 source columns; a surrogate patient_id was then added, confirming 5,000 unique patient rows.

Activity 3: Data Cleaning and Standardization

Identify and correct inconsistencies, invalid entries, formatting issues, and data quality problems in the Snowflake dataset, standardize values and formats across relevant columns, apply the required SQL-based cleaning and transformation steps, and validate the cleaned and standardized dataset to ensure consistency, accuracy, and readiness for further analysis and visualization.

Python code

```
# Step 1: standardise text and column names
df.columns = df.columns.str.strip().str.lower()
df["gender"] = df["gender"].str.strip().str.title()
print("Gender values:", df["gender"].value_counts().to_dict())

# Step 2: validity checks on units (BP mmHg, glucose mg/dL, BMI kg/m2)
limits = {"age": (18, 100), "bmi": (10, 60), "blood_pressure": (70, 250),
          "glucose": (40, 600), "cholesterol": (100, 400)}
for col, (lo, hi) in limits.items():
    bad = int((~df[col].between(lo, hi)).sum())
    print(f"{col:<15} range {df[col].min():>6} - {df[col].max():<6} out-of-range: {bad}")

# Step 3: binary flags must be 0/1
flags = ["smoking", "alcohol", "fatigue", "chest_pain", "dizziness",
         "heart_disease", "diabetes", "stroke"]
print("Non-binary flag values:", int((~df[flags].isin([0, 1])).sum().sum()))

# Step 4: feature engineering (matches the Tableau calculated fields)
df["age_bracket"] = pd.cut(df["age"], [0, 34, 50, 65, 200],
    labels=["18-34", "35-50", "51-65", "65+"])
df["bmi_category"] = pd.cut(df["bmi"], [0, 18.5, 24.95, 29.95, 100],
    labels=["Underweight", "Normal Weight", "Overweight", "Obese"], right=False)

bp  = df["blood_pressure"].apply(lambda x: 25 if x > 140 else 15 if x > 120 else 5)
glu = df["glucose"].apply(lambda x: 25 if x > 140 else 15 if x > 100 else 5)
chol = (df["cholesterol"] > 200).map({True: 20, False: 10})
bmi_pts = df["bmi"].apply(lambda x: 20 if x >= 30 else 10 if x >= 25 else 5)
smoke = df["smoking"] * 10
df["risk_index"] = bp + glu + chol + bmi_pts + smoke
df["risk_tier"] = pd.cut(df["risk_index"], [0, 49, 74, 200],
    labels=["Low Risk", "Moderate Risk", "High Risk"])

print(df["age_bracket"].value_counts().sort_index().to_dict())
print(df["bmi_category"].value_counts().to_dict())
print(df["risk_tier"].value_counts().to_dict())
print("health_risk_score unique values:", df["health_risk_score"].unique().tolist())
```

Snowflake SQL equivalent

```
-- Standardisation and validity checks
SELECT INITCAP(TRIM(gender))                              AS gender,
       COUNT(*)                                           AS patients,
       COUNT_IF(blood_pressure NOT BETWEEN 70 AND 250)    AS bad_bp,
       COUNT_IF(glucose        NOT BETWEEN 40 AND 600)    AS bad_glucose,
       COUNT_IF(bmi            NOT BETWEEN 10 AND 60)     AS bad_bmi
FROM STG_PATIENT_UNIQUE
GROUP BY 1;
```

Output

```
Gender values: {'Female': 2506, 'Male': 2494}
age             range     18 - 79     out-of-range: 0
bmi             range   15.0 - 42.6   out-of-range: 0
blood_pressure  range     90 - 179    out-of-range: 0
glucose         range     70 - 199    out-of-range: 0
cholesterol     range    150 - 299    out-of-range: 0
Non-binary flag values: 0
{'18-34': 1324, '35-50': 1301, '51-65': 1211, '65+': 1164}
{'Normal Weight': 2010, 'Overweight': 1671, 'Obese': 803, 'Underweight': 516}
{'Moderate Risk': 2771, 'High Risk': 1434, 'Low Risk': 795}
health_risk_score unique values: [100.0]
```

Output Summary: All values are within clinically plausible limits with standardized units (blood pressure in mmHg, glucose in mg/dL, BMI in kg/m²), and gender is standardized to Female / Male. Age brackets, WHO BMI categories and the Low / Moderate / High risk tier were derived with the same rules as the Tableau dashboards. One data-quality note: health_risk_score holds the single value 100 for every row, so it carries no information and the dashboards rely on the derived risk_index instead.

Activity 4: Data Transformation and Validation

Apply the required data transformations to prepare the cleaned data for analysis and modelling. Create necessary derived fields, apply business rules, filter or transform records where required, and validate the prepared data and other relevant quality checks before using it for the CURATED and MART layers.

Python code

```
# Validation checks before loading CLEAN_PATIENT_VITALS
checks = {
    "5,000 rows retained":        len(df) == 5000,
    "No NULLs in any column":     df.isnull().sum().sum() == 0,
    "patient_id unique":          df["patient_id"].is_unique,
    "All 4 age brackets present": df["age_bracket"].nunique() == 4,
    "All 4 BMI classes present":  df["bmi_category"].nunique() == 4,
    "All 3 risk tiers present":   df["risk_tier"].nunique() == 3,
}
for name, ok in checks.items():
    print(f"{'PASS' if ok else 'FAIL'}  {name}")

print("Final shape:", df.shape)
print("Mean risk index:", round(df["risk_index"].mean(), 1))
print(pd.crosstab(df["bmi_category"], df["age_bracket"]).to_string())

df.to_csv("clean_patient_vitals.csv", index=False)
```

Snowflake SQL equivalent

```
CREATE OR REPLACE TABLE CLEAN_PATIENT_VITALS AS
WITH scored AS (
  SELECT ROW_NUMBER() OVER (ORDER BY age, bmi, glucose) AS patient_id, s.*,
         CASE WHEN blood_pressure > 140 THEN 25 WHEN blood_pressure > 120 THEN 15 ELSE 5 END
       + CASE WHEN glucose > 140 THEN 25 WHEN glucose > 100 THEN 15 ELSE 5 END
       + CASE WHEN cholesterol > 200 THEN 20 ELSE 10 END
       + CASE WHEN bmi >= 30 THEN 20 WHEN bmi >= 25 THEN 10 ELSE 5 END
       + IFF(smoking = 1, 10, 0)                          AS risk_index
  FROM STG_PATIENT_UNIQUE s)
SELECT *,
  CASE WHEN age < 35 THEN '18-34' WHEN age <= 50 THEN '35-50'
       WHEN age <= 65 THEN '51-65' ELSE '65+' END        AS age_bracket,
  CASE WHEN bmi < 18.5 THEN 'Underweight' WHEN bmi <= 24.9 THEN 'Normal Weight'
       WHEN bmi <= 29.9 THEN 'Overweight' ELSE 'Obese' END AS bmi_category,
  CASE WHEN risk_index >= 75 THEN 'High Risk'
       WHEN risk_index >= 50 THEN 'Moderate Risk' ELSE 'Low Risk' END AS risk_tier
FROM scored;

SELECT risk_tier, COUNT(*) FROM CLEAN_PATIENT_VITALS GROUP BY 1;
```

Output

```
PASS  5,000 rows retained
PASS  No NULLs in any column
PASS  patient_id unique
PASS  All 4 age brackets present
PASS  All 4 BMI classes present
PASS  All 3 risk tiers present
Final shape: (5000, 21)
Mean risk index: 63.7
age_bracket    18-34  35-50  51-65  65+
bmi_category
Underweight      134    127    133  122
Normal Weight    552    524    485  449
Overweight       436    448    398  389
Obese            202    202    195  204
```

Output Summary: All six validation checks passed, and the derived counts reconcile with the Tableau dashboards (for example 1,434 High Risk, 2,771 Moderate Risk and 795 Low Risk patients). The cleaned 5,000 × 21 dataset is ready to stage from RAW_PATIENT_DATA into CLEAN_PATIENT_VITALS.

Outcomes: Major Data Preparation Findings

- **Zero NULL values across all 16 columns**: The null audit found no missing vital statistics, and the median-imputation safeguard leaves the cohort distribution unchanged, so no NULLs remain in the 5,000 records.

- **Dataset integrity preserved**: Clean 5,000 unique patient records were verified with no duplicate entries.
- **Feature engineering completed**: Continuous age and BMI were binned into discrete analytical tiers, and a composite risk index was classified into Low, Moderate and High Risk, matching the Tableau dashboard structure.

Figure 1: Health360 Data Preparation & ETL Processing Pipeline

